PYTHON BACKEND ENGINEERING

FastAPI — Build Real APIs

Classes 29 – 33 · Complete Student Notes

LEVEL
Designer → API Builder

TODAY
routes · Pydantic · SQLAlchemy · CRUD

INSIDE
5 Classes · Examples · Practice · 1 Capstone Project

Where we are. In Class 9 you learned that every framework is built out of classes, and you saw a tiny preview: `class UserCreate(BaseModel)`. Now we cash that in. Over the next five classes you will turn Python functions into a **web API** that a mobile app, a React frontend or another server can call over HTTP. You will describe data with **Pydantic schemas**, store it in a real database with **SQLAlchemy**, and finish with a complete set of **CRUD endpoints** — create, read, update, delete, with pagination and filtering. Every example here runs. Type it, run it, open `/docs`, and break it on purpose.

Class	Topic	You will be able to…
29	FastAPI Setup & First API	Install FastAPI, run a server, write endpoints with path & query params, use Swagger UI
30	Pydantic Schemas	Validate request bodies, shape responses, write nested and optional fields
31	Database Integration	Connect SQLAlchemy, define models, inject a DB session with `get_db()`
32	CRUD — Part 1	Create and read records with correct status codes and `HTTPException`
33	CRUD — Part 2	Update (PUT/PATCH), delete, paginate and filter lists

PART A · CLASS 29

FastAPI Setup & Your First API

1 What Is an API — and What Is HTTP?

An **API** (Application Programming Interface) is a door into your program that *other programs* can knock on. A **web API** uses **HTTP** as its language: a client sends a **request**, your server sends back a **response**. Your phone's bKash app, a React dashboard and a cron job on another server can all talk to the same API.



Every request has a **method** (what you want to do) and a **path** (which thing you want to do it to):

Method	Meaning	Has a body?	Example
GET	Read something. Never changes data.	No	`GET /products?category=books`
POST	Create something new	Yes	`POST /products` + JSON

PUT	Replace something completely	Yes	`PUT /products/7` + full JSON
PATCH	Change part of something	Yes	`PATCH /products/7` + `{"price": 900}`
DELETE	Remove something	Usually no	`DELETE /products/7`

> **JSON is the shared language**
>
> Python has dicts, JavaScript has objects, Java has maps — but over the wire everyone speaks **JSON**. `{"name": "Keyboard", "price": 2000}` looks almost exactly like a Python dict, and FastAPI converts between the two for you automatically. You return a dict (or an object); the client receives JSON.

2 Why FastAPI?

Python has several web frameworks. FastAPI became the default choice for new APIs because it uses something you already know — **type hints** — to do three jobs at once: validate input, convert types, and write documentation.

Feature	What it means for you
Type hints drive everything	Write `item_id: int` and FastAPI rejects `"abc"` before your code runs
Automatic docs	Swagger UI at `/docs` — a clickable test page generated from your code
Pydantic built in	Request/response validation with the classes you met in Class 9
Fast	Built on Starlette and supports `async`; one of the fastest Python frameworks
Dependency injection	Share things like a DB session cleanly across endpoints (Class 31)

Framework	Style	Best for
Flask	Minimal, you add everything yourself	Small apps, learning, legacy projects
Django	"Batteries included" — ORM, admin, templates	Full websites with HTML pages and an admin panel
FastAPI	API-first, typed, async-ready	JSON APIs, microservices, ML model serving

3 Installing FastAPI & Project Setup

Always work inside a **virtual environment** so each project has its own packages:

TERMINAL

```
mkdir shop-api && cd shop-api
python -m venv .venv                    # create the environment (once)

source .venv/bin/activate               # macOS / Linux
.venv\Scripts\activate                  # Windows

pip install "fastapi[standard]"         # FastAPI + uvicorn server + CLI + extras
pip freeze > requirements.txt           # record exact versions for teammates
```

> **What does `[standard]` add?**
>
> Plain `fastapi` is only the framework. `fastapi[standard]` also installs **uvicorn** (the server that actually listens for HTTP requests), the `fastapi` command-line tool, and helpers such as `email-validator` which you will need in Class 30. Quotes around it are required on macOS/zsh.

Start with one file. When the project grows (Class 32), you will split it into this structure:

TEXT - PROJECT LAYOUT - WHERE WE ARE HEADING

```
shop-api/
├── .venv/                  # virtual environment - never commit this
├── app/
│   ├── __init__.py
│   ├── main.py             # creates the FastAPI() app, includes routers
│   ├── database.py         # engine, session, Base, get_db()      (Class 31)
│   ├── models.py           # SQLAlchemy tables                    (Class 31)
│   ├── schemas.py          # Pydantic request/response shapes     (Class 30)
│   └── routers/
│       └── products.py     # the endpoints for one resource       (Class 32)
├── requirements.txt
└── .gitignore              # .venv/, __pycache__/, *.db
```

4 Your First Endpoint

PYTHON - MAIN.PY

```python
from fastapi import FastAPI

app = FastAPI(title="Shop API", version="1.0.0")    # the application object


@app.get("/")                                         # METHOD + PATH  ->  this function
def home():
    return {"message": "Welcome to Shop API"}         # dict  ->  JSON automatically


@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```
TERMINAL

fastapi dev main.py          # development server, auto-reloads when you save
# or the classic way:
uvicorn main:app --reload    # "main" = main.py, "app" = the variable inside it
```

Open `http://127.0.0.1:8000/` in a browser and you see `{"message":"Welcome to Shop API"}`. Here is what happened, step by step:

Step	What happens
1	The browser sends `GET /` to port 8000
2	Uvicorn receives the raw HTTP and hands it to your `app`
3	FastAPI looks up its route table: `GET` + `/` → `home()`
4	`home()` returns a dict; FastAPI converts it to JSON with status `200 OK`
5	Uvicorn sends the HTTP response back to the browser

> **The decorator is just registration**
>
> `@app.get("/health")` does not change your function — it *registers* it: "when a GET request for `/health` arrives, call this." There is one decorator per method: `@app.get`, `@app.post`, `@app.put`, `@app.patch`, `@app.delete`. A function registered this way is called an **endpoint**, a **route** or a **path operation**.

> ⚠️ **`fastapi dev` vs production**
>
> `fastapi dev` and `--reload` watch your files and restart on every save — great while coding, wasteful in production. On a real server you run `fastapi run main.py` (or `uvicorn main:app` without `--reload`), usually with several worker processes.

5 Path Parameters — Part of the URL

A **path parameter** is a variable piece of the URL that identifies *one specific thing*: `/products/7`, `/users/rahim`. Write it in curly braces and add a matching function parameter:

PYTHON - MAIN.PY

```python
from fastapi import FastAPI

app = FastAPI()

products = {
    1: {"name": "Keyboard", "price": 2000},
    2: {"name": "Mouse", "price": 1000},
}


@app.get("/products/{product_id}")
def get_product(product_id: int):          # the type hint is a RULE, not a comment
    return products.get(product_id, {"error": "not found"})   # Class 32 replaces this with a real 404
```

Request	What FastAPI does	Response
`GET /products/1`	Converts `"1"` (text in the URL) to `int` 1	`200` `{"name": "Keyboard", ...}`
`GET /products/9`	Valid int, but no such key	`200` `{"error": "not found"}`
`GET /products/abc`	Cannot convert to int — your function **never runs**	`422` validation error

RESPONSE - GET /PRODUCTS/ABC (422 UNPROCESSABLE CONTENT)

```json
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "product_id"],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "abc"
    }
  ]
}
```

Read that error like a sentence: "*in the **path**, the field **product_id** should be an integer, but you sent **abc**.*" You did not write a single `if` — the type hint did the work.

⚠️ Route order matters

FastAPI checks routes **top to bottom** and uses the first match. Put fixed paths **before** parameter paths:

```python
@app.get("/users/me")              # 1st - fixed path
def read_current_user():
    return {"user": "the logged-in user"}

@app.get("/users/{user_id}")       # 2nd - would also match "/users/me" if it came first
def read_user(user_id: int):
    return {"user_id": user_id}
```

Reverse the order and `GET /users/me` hits `read_user`, tries to turn `"me"` into an int, and returns 422.

🔬 Go Deeper — restricting a path param with `Enum`

When only a few values are valid, use an `Enum`. Swagger shows a dropdown, and anything else gets a 422:

```python
from enum import Enum

class Category(str, Enum):        # str + Enum -> values are strings, JSON-friendly
    books = "books"
    electronics = "electronics"
    grocery = "grocery"

@app.get("/categories/{category}")
def products_in(category: Category):
    return {"category": category.value}    # GET /categories/toys -> 422
```

6 Query Parameters — Options After the `?`

A **query parameter** comes after `?` in the URL as `key=value` pairs joined by `&`: `/products?category=books&max_price=500`. In FastAPI, **any function parameter that is not in the path is a query parameter**. Give it a default and it becomes optional:

PYTHON - MAIN.PY

```python
from fastapi import FastAPI

app = FastAPI()

catalog = [
    {"id": 1, "name": "Keyboard", "category": "electronics", "price": 2000, "in_stock": True},
    {"id": 2, "name": "Mouse", "category": "electronics", "price": 1000, "in_stock": False},
    {"id": 3, "name": "Python Book", "category": "books", "price": 850, "in_stock": True},
]


@app.get("/products")
def list_products(
    category: str | None = None,      # optional - None means "not given"
    max_price: float | None = None,
    in_stock: bool = False,           # optional with a default
):
    results = catalog
    if category is not None:
        results = [p for p in results if p["category"] == category]
    if max_price is not None:
        results = [p for p in results if p["price"] <= max_price]
    if in_stock:
        results = [p for p in results if p["in_stock"]]
    return results
```

URL	Result
`/products`	All 3 products
`/products?category=electronics`	Keyboard, Mouse
`/products?category=electronics&in_stock=true`	Keyboard only
`/products?max_price=900`	Python Book
`/products?max_price=cheap`	`422` — `"cheap"` is not a float

> **bool query params are forgiving**
>
> For a `bool`, FastAPI accepts `true/false`, `1/0`, `yes/no`, `on/off` (any capitalisation). `?in_stock=yes` becomes `True`.

Adding rules with `Query()` and `Annotated`

A type alone says "must be an int". Often you also need "between 1 and 100" or "at least 2 characters". Wrap the type in `Annotated[...]` and attach a `Query()` with the rules:

PYTHON

```python
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/search")
def search(
    q: Annotated[str, Query(min_length=2, max_length=50)],          # REQUIRED (no default)
    limit: Annotated[int, Query(ge=1, le=100)] = 10,                # 1 <= limit <= 100
    sort: Annotated[str, Query(pattern="^(price|name)$")] = "name", # only two values allowed
):
    return {"q": q, "limit": limit, "sort": sort}

# /search?q=k              -> 422  (String should have at least 2 characters)
# /search?q=key&limit=500  -> 422  (Input should be less than or equal to 100)
# /search                  -> 422  (Field required - q has no default)
```

Rule	Meaning	Works on
`gt` / `ge`	greater than / greater than or equal	numbers
`lt` / `le`	less than / less than or equal	numbers
`min_length` / `max_length`	length limits	strings, lists
`pattern`	must match a regular expression	strings
`alias`	different name in the URL (`?item-query=`)	any
`description`	shown in Swagger UI	any

Path vs Query — which one?

	Path parameter	Query parameter
Looks like	`/products/7`	`/products?category=books`
Purpose	**Identifies** one resource	**Filters, sorts, paginates** or tweaks the result
Required?	Always	Optional if it has a default
Question it answers	"Which one?"	"Which ones, and how?"
Combined	`GET /users/5/orders?status=paid&limit=10` — orders of user 5, only paid ones, max 10	

7 Automatic Docs — Swagger UI

Run your app and open `http://127.0.0.1:8000/docs`. FastAPI has read your function signatures and built an interactive page: every endpoint, every parameter, its type and rules. Click **Try it out →**

Execute and it sends a real request. You will use this page more than the browser address bar.

URL	What you get
`/docs`	Swagger UI — interactive, test endpoints in the browser
`/redoc`	ReDoc — clean, read-only documentation for clients
`/openapi.json`	The raw **OpenAPI** spec — the machine-readable description both pages are built from

Make the docs more useful with tags, summaries and docstrings:

PYTHON

```python
@app.get("/products", tags=["Products"], summary="List products")
def list_products(category: str | None = None):
    """
    Return products, optionally filtered by **category**.
    This docstring appears in Swagger UI (Markdown works).
    """
    return []
```

> **Why OpenAPI matters at work**
>
> The frontend team reads `/docs` instead of asking you what fields an endpoint needs. Tools can even **generate client code** (TypeScript, Kotlin, Swift) from `/openapi.json`. Your code *is* the documentation, so it can never go out of date.

PRACTICE 29

1. Create `main.py` with `GET /` returning your name and `GET /health` returning `{"status": "ok"}`. Run it with `fastapi dev`.
2. Make an in-memory list of 5 students (`id, name, dept, cgpa`). Add `GET /students/{student_id}`. Try `/students/abc` and read the 422 error out loud.
3. Add `GET /students` with optional query params `dept` and `min_cgpa` (between 0 and 4, use `Query`). Test every combination in `/docs`.
4. Add `GET /students/top` returning the highest CGPA. Place it **after** `/students/{student_id}`, see what breaks, then fix the order.

PART B · CLASS 30

Pydantic Schemas — the Shape of Your Data

8 Why Do We Need Schemas?

Path and query params carry small values. To **create** something, the client sends a whole JSON object in the request **body**. The naive approach — accept any dict — repeats every problem from Class 9's procedural bank:

PYTHON - THE NAIVE WAY - DON'T DO THIS

```python
from fastapi import FastAPI, Body

app = FastAPI()

@app.post("/products")
def create_product(data: dict = Body()):   # "any dict" - zero protection
    # what if "price" is missing?  what if it is "two thousand"?  or -500?
    # what if the client sends "prise" (typo)?  every check is now YOUR job,
    # written by hand, in every endpoint, and Swagger has no idea what to send.
    return data
```

A **schema** is a class that describes exactly what a valid piece of data looks like. It is the **contract** between your API and its clients: "send me this shape, I will send you that shape." In FastAPI, schemas are **Pydantic models** — classes that inherit from `BaseModel`.

📥 Without a schema

- Manual `if` checks in every endpoint
- Wrong types crash deep inside your code
- Swagger shows an empty "object"
- Clients guess field names

✅ With a schema

- Validation happens **before** your function runs
- Types converted: `"25"` → `25`
- Swagger shows every field, type and example
- Clear 422 errors that name the bad field

9 `BaseModel` & the Request Body

Declare a class with typed fields — exactly like a dataclass from Class 9 — then use it as a parameter type. FastAPI sees a Pydantic model and knows: "*this comes from the JSON body.*"

PYTHON - MAIN.PY

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class ProductCreate(BaseModel):          # the SHAPE a client must send
    name: str
    price: float
    stock: int = 0                       # default -> optional in the JSON
    category: str = "general"


@app.post("/products")
def create_product(product: ProductCreate):     # body is parsed + validated here
    print(type(product))                         # <class 'main.ProductCreate'> - an OBJECT
    total_value = product.price * product.stock  # dot access, real types
    return {"received": product, "stock_value": total_value}
```

JSON body sent	Result
`{"name": "Keyboard", "price": 2000, "stock": 5}`	`200` — `category` filled with `"general"`
`{"name": "Keyboard", "price": "2000"}`	`200` — `"2000"` converted to `2000.0`
`{"name": "Keyboard"}`	`422` — `price` : Field required
`{"name": "Keyboard", "price": "cheap"}`	`422` — `price` : Input should be a valid number

How FastAPI decides where each parameter comes from — memorise this table:

Parameter declared as…	FastAPI reads it from…	Example
Name appears in the path `{...}`	The URL path	`product_id: int`
Simple type (`int` **,** `str` **,** `bool` **…), not in path**	The query string	`limit: int = 10`
A Pydantic `BaseModel`	The JSON request body	`product: ProductCreate`

You can mix all three in one endpoint:

PYTHON

```python
@app.put("/stores/{store_id}/products/{product_id}")
def update_in_store(store_id: int, product_id: int,      # path
                    product: ProductCreate,              # body
                    notify: bool = False):               # query
    return {"store": store_id, "product": product_id, "data": product, "notify": notify}
```

Pydantic works without FastAPI too

Pydantic is just a library. Try it in a plain Python shell to understand it:

PYTHON

```python
from pydantic import BaseModel, ValidationError

class User(BaseModel):
    name: str
    age: int

u = User(name="Rahim", age="25")         # "25" -> 25
print(u.age + 1)                         # 26
print(u.model_dump())                    # {'name': 'Rahim', 'age': 25}  <- back to a dict
print(u.model_dump_json())               # {"name":"Rahim","age":25}    <- JSON text

u2 = User.model_validate({"name": "Karim", "age": 30})   # build from a dict
try:
    User(name="Hasan", age="old")
except ValidationError as e:
    print(e.error_count(), "error")      # 1 error
```

10 Field Validation — Rules on Every Field

A type says *what kind* of value. `Field()` adds *which values are acceptable* — the same rule names you used with `Query()`:

PYTHON - SCHEMAS.PY

```python
from pydantic import BaseModel, EmailStr, Field


class UserCreate(BaseModel):
    name: str = Field(min_length=2, max_length=50)
    email: EmailStr                                  # real email format check
    age: int = Field(ge=18, le=100)                  # 18 <= age <= 100
    phone: str = Field(pattern=r"^01[3-9]\d{8}$")    # Bangladeshi mobile number
    password: str = Field(min_length=8)
    bio: str = Field(default="", max_length=300)


class ProductCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100, examples=["Mechanical Keyboard"])
    price: float = Field(gt=0, le=1_000_000, description="Price in Taka")
    stock: int = Field(default=0, ge=0)
```

Send `{"name": "K", "email": "rahim", "age": 15, "phone": "123", "password": "abc"}` and you get **all** the problems at once — Pydantic does not stop at the first error:

RESPONSE - 422 (SHORTENED)

```
{
  "detail": [
    {"loc": ["body", "name"],     "msg": "String should have at least 2 characters"},
    {"loc": ["body", "email"],    "msg": "value is not a valid email address: An email address must
have an @-sign."},
    {"loc": ["body", "age"],      "msg": "Input should be greater than or equal to 18"},
    {"loc": ["body", "phone"],    "msg": "String should match pattern '^01[3-9]\\d{8}$'"},
    {"loc": ["body", "password"], "msg": "String should have at least 8 characters"}
  ]
}
```

Custom rules with `@field_validator`

When a rule cannot be expressed with `gt` / `max_length` / `pattern`, write a method. Raise `ValueError` to reject; **return** the value (possibly cleaned) to accept:

PYTHON

```python
from pydantic import BaseModel, Field, field_validator, model_validator


class SignUp(BaseModel):
    username: str = Field(min_length=3)
    password: str
    confirm_password: str

    @field_validator("username")
    @classmethod
    def clean_username(cls, value: str) -> str:
        if " " in value:
            raise ValueError("username cannot contain spaces")
        return value.lower()                    # accept, but normalise: "Rahim" -> "rahim"

    @field_validator("password")
    @classmethod
    def strong_password(cls, value: str) -> str:
        if len(value) < 8 or not any(ch.isdigit() for ch in value):
            raise ValueError("password needs 8+ characters and at least one digit")
        return value

    @model_validator(mode="after")              # runs after ALL fields are valid
    def passwords_match(self):
        if self.password != self.confirm_password:
            raise ValueError("passwords do not match")
        return self
```

Tool	Use when	Example
Type hint	The kind of value	`age: int`
`Field(...)`	Limits on one field	`Field(ge=18)`
Special types	Common formats	`EmailStr`, `HttpUrl`, `datetime`, `UUID`

`@field_validator`	Custom logic on one field	no spaces, normalise case
`@model_validator`	A rule involving several fields	passwords match, start < end date

11 Optional Fields, Defaults and `None`

Three different ideas are often confused. Look at each field below carefully:

```python
from pydantic import BaseModel


class Profile(BaseModel):
    name: str                       # 1. REQUIRED, cannot be null
    city: str = "Dhaka"             # 2. OPTIONAL, has a default, cannot be null
    nickname: str | None = None     # 3. OPTIONAL, may be null, default null
    website: str | None             # 4. REQUIRED, but null is an allowed value


print(Profile(name="Rahim", website=None))
# name='Rahim' city='Dhaka' nickname=None website=None
```

Declaration	Must the client send it?	Can it be `null` ?
`name: str`	Yes	No
`city: str = "Dhaka"`	No (default used)	No
`nickname: str | None = None`	No	Yes
`website: str | None`	Yes — even if just `null`	Yes

> **`Optional[str]` is the old spelling**
>
> You will see `from typing import Optional` and `Optional[str] = None` in older tutorials. It means exactly the same as `str | None = None` . The `|` form is modern (Python 3.10+) and what we use. Note: `Optional` only means "may be None" — it is the **default value** that makes a field not required.

12 Response Models — Control What Goes Out

Validating *input* is half the job. You must also control *output*. The classic disaster: returning a user object that still contains the password. A **response model** filters and validates whatever your function returns:

PYTHON - MAIN.PY

```python
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()


class UserCreate(BaseModel):        # what the client SENDS
    name: str
    email: EmailStr
    password: str


class UserOut(BaseModel):           # what the client RECEIVES - no password field
    id: int
    name: str
    email: EmailStr


fake_db: list[dict] = []


@app.post("/users", response_model=UserOut)
def create_user(user: UserCreate):
    record = {"id": len(fake_db) + 1, **user.model_dump()}   # contains the password!
    fake_db.append(record)
    return record                   # FastAPI keeps only id, name, email


@app.get("/users", response_model=list[UserOut])   # a list of them
def list_users():
    return fake_db
```

RESPONSE - POST /USERS

```
{"id": 1, "name": "Rahim", "email": "rahim@mail.com"}
```

You can also declare the response model as the **return type**: `def create_user(user: UserCreate) -> UserOut:`. Both work; `response_model=` is needed when you return a dict or ORM object that is not literally a `UserOut` — which is exactly our case — so we use it throughout these notes.

One resource, several schemas

Real projects keep a small **family** of schemas per resource. Inheritance (Class 9!) removes the repetition:

PYTHON - SCHEMAS.PY

```python
from datetime import datetime
from pydantic import BaseModel, Field


class ProductBase(BaseModel):                    # fields shared by everyone
    name: str = Field(min_length=2, max_length=100)
    price: float = Field(gt=0)
    category: str = "general"


class ProductCreate(ProductBase):                # POST body - client sends this
    stock: int = Field(default=0, ge=0)


class ProductUpdate(BaseModel):                  # PATCH body - every field optional (Class 33)
    name: str | None = Field(default=None, min_length=2, max_length=100)
    price: float | None = Field(default=None, gt=0)
    category: str | None = None
    stock: int | None = Field(default=None, ge=0)


class ProductOut(ProductBase):                   # response - adds server-generated fields
    id: int
    stock: int
    created_at: datetime
```

Schema	Used for	Contains
`XBase`	Shared fields (never used directly in endpoints)	Fields common to all
`XCreate`	POST request body	What the client may set; no `id`
`XUpdate`	PATCH request body	Same fields, all optional
`XOut`	`response_model`	Safe fields + `id`, timestamps; never secrets

13 Nested Schemas — Objects Inside Objects

Composition from Class 9 works in schemas too: a field's type can be another model, or a list of models. Validation goes all the way down.

PYTHON - MAIN.PY

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Address(BaseModel):
    area: str
    city: str = "Dhaka"
    postcode: str = Field(pattern=r"^\d{4}$")


class OrderItem(BaseModel):
    product_id: int
    qty: int = Field(ge=1, le=20)


class OrderCreate(BaseModel):
    customer: str
    address: Address                                  # one nested object
    items: list[OrderItem] = Field(min_length=1)      # a list of nested objects, at least 1
    note: str | None = None


@app.post("/orders")
def place_order(order: OrderCreate):
    total_qty = sum(item.qty for item in order.items)   # real objects all the way down
    return {"deliver_to": order.address.area, "items": len(order.items), "total_qty": total_qty}
```

JSON - REQUEST BODY

```json
{
  "customer": "Rahim",
  "address": {"area": "Mirpur 10", "postcode": "1216"},
  "items": [
    {"product_id": 1, "qty": 2},
    {"product_id": 3, "qty": 1}
  ]
}
```

Response: `{"deliver_to": "Mirpur 10", "items": 2, "total_qty": 3}`. Send `"qty": 0` in the second item and the error location tells you exactly where: `["body", "items", 1, "qty"]`.

🔬 Go Deeper — the Pydantic v2 method names

Pydantic v2 (the current version) renamed most methods. Old tutorials use v1 names — know both:

Pydantic v2 (use this)	Pydantic v1 (old)	Does
`obj.model_dump()`	`obj.dict()`	model → dict
`obj.model_dump(exclude_unset=True)`	`obj.dict(exclude_unset=True)`	only fields the client actually sent (PATCH!)
`Model.model_validate(data)`	`Model.parse_obj(data)`	dict / object → model
`model_config = ConfigDict(from_attributes=True)`	`class Config: orm_mode = True`	read from ORM objects (Class 31)
`@field_validator`	`@validator`	custom field rule

PRACTICE 30

1. Write `StudentCreate`: `name` (2–50 chars), `email` (`EmailStr`), `dept` (only `CSE|EEE|BBA` — use a `pattern` or an `Enum`), `cgpa` (0–4), optional `phone`.
2. Write `StudentOut` with an `id` and **without** email. Build `POST /students` and `GET /students` using an in-memory list and `response_model`.
3. Add a `@field_validator` that title-cases the name (`"rahim uddin"` → `"Rahim Uddin"`).
4. Add a nested `guardian: Guardian` (name, phone) and send a body with a bad guardian phone. Read the `loc` in the 422.

PART C · CLASS 31

Database Integration — SQLAlchemy + FastAPI

14 Why a Database?

So far our data lived in a Python list. Stop the server and it is gone. Run two server processes and each has its own different list. A **database** stores data on disk, survives restarts, is shared by every process, and can search millions of rows quickly.

Storage	Survives restart?	Shared between processes?	Fast search?
Python list / dict	No	No	Only for small data
JSON / CSV file	Yes	Unsafe — two writers corrupt it	No — read the whole file
SQL database	Yes	Yes, with transactions	Yes — indexes

We use **SQLite** in class (a single file, nothing to install) and the exact same code later works with **PostgreSQL** by changing one line — the connection URL.

15 SQLAlchemy — Python Classes as Tables

SQLAlchemy is an **ORM** (Object-Relational Mapper). Instead of writing SQL strings by hand, you write Python classes; SQLAlchemy translates. It is OOP from Class 9 applied to databases:

Database world	Python / SQLAlchemy world
Table `products`	Class `Product(Base)`
Column `price`	Attribute `price: Mapped[float]`
One row	One object: `Product(name="Mouse", price=1000)`
`INSERT INTO products ...`	`db.add(product)` + `db.commit()`
`SELECT * FROM products WHERE id = 7`	`db.get(Product, 7)`

Three objects do all the work. Learn their jobs:

⚙️ Engine

The connection to the database

Created **once** per app

`create_engine(URL)`

🗂️ Session

A "workspace" for one unit of work

One **per request**

add · get · commit · close

🧱 Base + Models

Your table classes

Inherit from `Base`

`class Product(Base)`

TERMINAL

```
pip install sqlalchemy        # already pulled in? check with: pip show sqlalchemy
```

16 database.py — Engine, Session, Base

PYTHON - APP/DATABASE.PY

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

DATABASE_URL = "sqlite:///./shop.db"            # a file called shop.db next to where you run
# DATABASE_URL = "postgresql+psycopg://user:pass@localhost:5432/shop"   # later, in production

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},   # SQLite-only: allow use from FastAPI's threads
    echo=False,                                   # True = print every SQL statement (great for
learning)
)

SessionLocal = sessionmaker(bind=engine, autoflush=False)   # a FACTORY that makes sessions


class Base(DeclarativeBase):                      # every model inherits from this
    pass
```

The connection URL

Format: `dialect+driver://user:password@host:port/database` . For SQLite it is just a file path: `sqlite:///./shop.db` (three slashes = relative path). **Never** hard-code real passwords — in real projects the URL comes from an environment variable.

17 `models.py` — Defining Tables

PYTHON - APP/MODELS.PY

```python
from datetime import datetime, timezone

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


def utc_now():
    return datetime.now(timezone.utc)


class Product(Base):
    __tablename__ = "products"                                  # the real table name

    id: Mapped[int] = mapped_column(primary_key=True)           # auto-increment id
    name: Mapped[str] = mapped_column(String(100), index=True)  # VARCHAR(100), indexed
    sku: Mapped[str] = mapped_column(String(20), unique=True)   # no two rows may share it
    price: Mapped[float]                                        # NOT NULL float
    stock: Mapped[int] = mapped_column(default=0)
    description: Mapped[str | None]                             # "| None" -> column may be NULL
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=utc_now) # a FUNCTION, called per row

    def __repr__(self):
        return f"Product(id={self.id}, name={self.name!r})"
```

Piece	Meaning
`Mapped[int]`	The Python type; also decides NOT NULL (plain type) vs NULL allowed (`| None`)
`mapped_column(...)`	Extra column settings — only needed when you have some
`primary_key=True`	Unique row id; integer primary keys auto-increment
`unique=True` / `index=True`	Database-level uniqueness / faster lookups on this column
`default=...`	Value used when you don't set it. Pass a **function** (`utc_now`), not a call (`utc_now()`)

> ⚠️ **`default=utc_now()` — the frozen timestamp bug**
>
> With parentheses, the function runs **once, when the class is defined**, and every row gets the same server-start time. Without parentheses, SQLAlchemy calls it fresh for every new row. Same trap as Class 3's `def f(items=[])`.

Model vs Schema — the most important distinction in this course

Beginners mix these up constantly. You now have **two** kinds of class for the same "product":

	SQLAlchemy Model	Pydantic Schema
File	`models.py`	`schemas.py`
Inherits from	`Base` (DeclarativeBase)	`BaseModel`
Describes	How data is **stored** (a table)	How data **travels** over HTTP
Talks to	The database	The client
How many?	One per table	Several per resource (Create, Update, Out)
Example difference	Has `hashed_password`, `id`, `created_at`	`UserCreate` has `password`; `UserOut` has neither

18 Creating the Tables

`Base.metadata` knows every class that inherits from `Base`. `create_all` issues `CREATE TABLE` for any table that does not exist yet. Run it once at startup using a **lifespan** function:

PYTHON - APP/MAIN.PY

```python
from contextlib import asynccontextmanager

from fastapi import FastAPI

from app import models                      # importing registers the models on Base
from app.database import Base, engine


@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)    # runs ONCE, before the first request
    yield                                    # the app serves requests here
    # code after yield runs on shutdown (close pools, flush logs...)


app = FastAPI(title="Shop API", lifespan=lifespan)
```

> **`create_all` does not update tables**
>
> If `products` already exists and you add a new column to the model, `create_all` does nothing. While learning, delete `shop.db` and restart. In real projects a migration tool (**Alembic**) applies changes to existing tables — you will meet it in a later class.

19 Dependency Injection & the `get_db()` Pattern

Every request that touches the database needs a session — and that session **must be closed** afterwards, even if the endpoint crashes. Copy-pasting open/close code into every endpoint is the

Class 9 "duplication" smell. FastAPI's answer is **dependency injection**: you write the setup/cleanup once, and FastAPI "injects" the result into any endpoint that asks for it.

PYTHON - APP/DATABASE.PY (ADD AT THE BOTTOM)

```python
from typing import Annotated
from fastapi import Depends
from sqlalchemy.orm import Session


def get_db():
    db = SessionLocal()          # 1. open a session for THIS request
    try:
        yield db                 # 2. hand it to the endpoint; pause here while it runs
    finally:
        db.close()               # 3. always runs - success OR exception


SessionDep = Annotated[Session, Depends(get_db)]   # a reusable type: "a Session from get_db"
```

PYTHON - USING IT IN AN ENDPOINT

```python
from sqlalchemy import select
from app.database import SessionDep
from app.models import Product


@app.get("/products/count")
def count_products(db: SessionDep):            # FastAPI calls get_db() for you
    return {"count": len(db.scalars(select(Product)).all())}
```

1. Request arrives GET /products/count → **2. FastAPI sees** `db: SessionDep` calls `get_db()` → **3. get_db yields** a fresh Session → **4. Endpoint runs** uses `db` → **5. After response** `finally:` closes it

Why `yield` and not `return`?

`return db` would end the function, and there would be nowhere to put the cleanup. `yield` pauses `get_db` in the middle: FastAPI runs your endpoint, then resumes `get_db` so the `finally` block closes the session. One session per request, always closed — the leak is impossible.

🔬 Go Deeper — dependencies are not only for databases

Any function can be a dependency, and dependencies can depend on other dependencies. Common uses: reading the current user from a token, shared pagination params, settings, API keys.

PYTHON

```python
from typing import Annotated
from fastapi import Depends, Query

def pagination(skip: Annotated[int, Query(ge=0)] = 0,
               limit: Annotated[int, Query(ge=1, le=100)] = 20):
    return {"skip": skip, "limit": limit}

PageDep = Annotated[dict, Depends(pagination)]

@app.get("/orders")
def list_orders(page: PageDep):         # skip & limit appear as query params in /docs
    return page                         # {"skip": 0, "limit": 20}
```

In tests you can swap a dependency for a fake with `app.dependency_overrides[get_db] = get_test_db` — that is the "pass dependencies in" habit from Class 9, Section 20.

20 Connecting Model ↔ Schema: `from_attributes`

Your endpoint gets a SQLAlchemy `Product` object from the database but must return a Pydantic `ProductOut`. Pydantic normally reads **dicts** (`data["name"]`). Tell it to also read **attributes** (`obj.name`):

PYTHON - APP/SCHEMAS.PY

```python
from datetime import datetime
from pydantic import BaseModel, ConfigDict


class ProductOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)   # read obj.attr, not only dict["key"]

    id: int
    name: str
    sku: str
    price: float
    stock: int
    created_at: datetime
```

Now an endpoint can simply `return product` (the ORM object) with `response_model=ProductOut`, and FastAPI converts it — dropping any column you did not list. Forget `from_attributes` and you get a validation error on the **response** (a 500, because it is your bug, not the client's).

🔬 Go Deeper — a first look at relationships (foreign keys)

Tables often point at each other: a product belongs to a category. A `ForeignKey` stores the link; `relationship()` lets you walk it as a Python attribute. A nested schema can then return the related object.

PYTHON

```python
from sqlalchemy import ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship

class Category(Base):
    __tablename__ = "categories"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    products: Mapped[list["Product"]] = relationship(back_populates="category")

class Product(Base):
    __tablename__ = "products"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    category_id: Mapped[int] = mapped_column(ForeignKey("categories.id"))   # the stored link
    category: Mapped[Category] = relationship(back_populates="products")    # the Python
shortcut

# product.category.name   -> "Electronics"      category.products -> [Product, Product, ...]

class CategoryBrief(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str

class ProductOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    category: CategoryBrief          # nested schema filled from product.category
```

You will need this pattern in the project.

21 Async Sessions — an Introduction

A normal (**sync**) database call makes the worker *wait* while the database answers. FastAPI runs sync endpoints in a thread pool, so this works well for most apps. With **async**, the worker can serve other requests during that wait — useful when you have very many concurrent, I/O-heavy requests.

PYTHON - ASYNC VERSION (PIP INSTALL AIOSQLITE)

```python
from typing import Annotated
from fastapi import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

engine = create_async_engine("sqlite+aiosqlite:///./shop.db")          # async DRIVER in the URL
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)


async def get_db():
    async with AsyncSessionLocal() as session:      # "async with" closes it for us
        yield session

AsyncSessionDep = Annotated[AsyncSession, Depends(get_db)]


@app.get("/products")
async def list_products(db: AsyncSessionDep):       # async def
    result = await db.scalars(select(Product))      # every DB call is awaited
    return result.all()

# creating tables needs an async connection:
#   async with engine.begin() as conn:
#       await conn.run_sync(Base.metadata.create_all)
```

	Sync (what we use in class)	Async
Endpoint	`def`	`async def`
Engine / session	`create_engine`, `Session`	`create_async_engine`, `AsyncSession`
Driver	`sqlite`, `psycopg`	`aiosqlite`, `asyncpg` / `psycopg` async
DB calls	`db.commit()`	`await db.commit()`
Best for	Learning, most CRUD apps	High concurrency, many slow I/O calls

⚠️ Never call blocking code inside `async def`

A sync session (or `time.sleep`, or `requests.get`) inside an `async def` endpoint blocks the **whole event loop** — every other request freezes. Rule: **sync session → `def` endpoints**, **async session → `async def` endpoints**. Don't mix them.

PRACTICE 31

1. Create `app/database.py` and `app/models.py` with a `Student` model (`id, name, email` unique, `dept` , `cgpa` , `created_at`).
2. Create the tables with a lifespan function. Open `students.db` in "DB Browser for SQLite" (or the VS Code SQLite extension) and find your table.
3. Set `echo=True` on the engine, restart, and read the `CREATE TABLE` SQL that SQLAlchemy wrote for you.
4. Add `get_db()` and an endpoint `GET /students/count` that uses `SessionDep` . Put a `print("closing")` in the `finally` and watch it run after every request.

PART D · CLASS 32

CRUD Endpoints — Part 1: Create & Read

22 CRUD — Four Verbs Behind Every App

Almost every backend feature is one of four operations on some resource. Facebook posts, bKash transactions, Daraz products — all **CRUD**. A well-designed API maps them to HTTP methods consistently, so a client can guess your endpoints without reading the docs.

Operation	HTTP	Path	SQL	Success code
Create	POST	`/products`	`INSERT`	`201 Created`
Read all	GET	`/products`	`SELECT ... LIMIT`	`200 OK`
Read one	GET	`/products/{id}`	`SELECT ... WHERE id=`	`200 OK`
Update (full)	PUT	`/products/{id}`	`UPDATE`	`200 OK`
Update (partial)	PATCH	`/products/{id}`	`UPDATE`	`200 OK`
Delete	DELETE	`/products/{id}`	`DELETE`	`204 No Content`

> **REST naming rules**
>
> Paths are **nouns, plural**: `/products`, not `/getProducts` or `/product/create`. The HTTP method already says the verb. Nest to show ownership: `/users/5/orders`. Use lowercase and hyphens: `/order-items`.

23 Status Codes — Tell the Client What Happened

The status code is the first thing a client checks. A frontend shows a green toast on `201`, a "not found" page on `404`, and red text under form fields on `422`. Returning `200` with `{"error": "..."}` (like Class 29's quick hack) forces every client to read your body to discover failure — don't.

Code	Name	Use it when…	Who raises it
200	OK	Successful GET, PUT, PATCH	FastAPI default
201	Created	POST created a new resource	You: `status_code=201`
204	No Content	DELETE succeeded, nothing to return	You: `status_code=204`
400	Bad Request	Request is valid JSON but breaks a business rule	You

401 / 403	Unauthorized / Forbidden	Not logged in / logged in but not allowed	You (auth classes)
404	Not Found	The id does not exist	You
409	Conflict	Duplicate email/SKU, state conflict	You
422	Unprocessable Content	Body/params fail schema validation	FastAPI, automatically
500	Internal Server Error	Your code crashed — always a bug to fix	Nobody on purpose

Families: **2xx** success · **4xx** the *client* did something wrong · **5xx** the *server* did something wrong. Use the named constants for readability: `from fastapi import status` → `status.HTTP_201_CREATED`, `status.HTTP_404_NOT_FOUND`.

24 Setting Up the Product API

We build on the `database.py` and `models.py` from Class 31. Here are the schemas for this part:

PYTHON - APP/SCHEMAS.PY

```python
from datetime import datetime
from pydantic import BaseModel, ConfigDict, Field


class ProductCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    sku: str = Field(min_length=3, max_length=20)
    price: float = Field(gt=0)
    stock: int = Field(default=0, ge=0)
    description: str | None = None


class ProductOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    sku: str
    price: float
    stock: int
    description: str | None
    is_active: bool
    created_at: datetime
```

25 POST — Create

PYTHON - APP/MAIN.PY (ENDPOINT)

```python
from fastapi import HTTPException, status
from sqlalchemy import select

from app.database import SessionDep
from app.models import Product
from app.schemas import ProductCreate, ProductOut


@app.post("/products", response_model=ProductOut, status_code=status.HTTP_201_CREATED)
def create_product(data: ProductCreate, db: SessionDep):
    # 1. business rule: SKU must be unique -> 409 instead of a database crash
    existing = db.scalar(select(Product).where(Product.sku == data.sku))
    if existing:
        raise HTTPException(status_code=409, detail=f"SKU {data.sku} already exists")

    product = Product(**data.model_dump())   # 2. schema -> model  (dict unpacked into kwargs)
    db.add(product)                          # 3. stage it in the session
    db.commit()                              # 4. write to the database (INSERT runs here)
    db.refresh(product)                      # 5. reload: now has id, created_at, defaults
    return product                           # 6. model -> ProductOut -> JSON
```

Call	What it does	Forget it and…
`db.add(obj)`	Tells the session "this object should be saved"	Nothing is ever saved
`db.commit()`	Runs the SQL and makes it permanent (transaction)	Data vanishes when the session closes
`db.refresh(obj)`	Re-reads the row, filling `id` and DB defaults	Usually still works, but don't rely on it
`db.rollback()`	Undo everything since the last commit	A failed transaction poisons the session

🔬 Go Deeper — check first, or catch `IntegrityError`?

The "check, then insert" above has a tiny race: two requests with the same SKU at the same millisecond can both pass the check. The database's `unique=True` still protects you — the second `commit()` raises `sqlalchemy.exc.IntegrityError`. Production code often does both:

PYTHON

```python
from sqlalchemy.exc import IntegrityError

try:
    db.commit()
except IntegrityError:
    db.rollback()                    # clean the failed transaction
    raise HTTPException(409, "SKU already exists")
```

26 GET — Read All & Read One

PYTHON - APP/MAIN.PY (ENDPOINTS)

```python
@app.get("/products", response_model=list[ProductOut])
def list_products(db: SessionDep):
    stmt = select(Product).order_by(Product.id)   # build the query (nothing runs yet)
    return db.scalars(stmt).all()                 # run it -> list of Product objects


@app.get("/products/{product_id}", response_model=ProductOut)
def read_product(product_id: int, db: SessionDep):
    product = db.get(Product, product_id)         # lookup by primary key; None if missing
    if product is None:
        raise HTTPException(status_code=404, detail=f"Product {product_id} not found")
    return product
```

SQLAlchemy 2.0 query	Returns	Use for
`db.get(Product, 7)`	One object or `None`	Lookup by primary key
`db.scalars(select(Product)).all()`	List of objects	Many rows
`db.scalar(select(Product).where(...))`	First object or `None`	Find by another unique column
`db.scalar(select(func.count()).select_from(Product))`	An int	Counting rows

> **`db.query(...)` is the old style**
>
> Many tutorials write `db.query(Product).filter(Product.id == 7).first()`. That is the SQLAlchemy 1.x API — it still works, but new code uses `select()` + `db.scalars()` / `db.get()`. Recognise the old one, write the new one.

27 `HTTPException` & Error Responses

`raise HTTPException(...)` is the `raise ValueError` of Class 9, but for HTTP: it stops the endpoint immediately and FastAPI turns it into a JSON error response with the status you chose.

PYTHON

```python
raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="Product 99 not found",              # a string, or any JSON-able dict/list
    headers={"X-Error-Code": "PRODUCT_NOT_FOUND"},  # optional extra headers
)
```

RESPONSE - GET /PRODUCTS/99 (404 NOT FOUND)

```
{"detail": "Product 99 not found"}
```

Avoid repeating "fetch or 404" in every endpoint — a small helper keeps it in one place:

PYTHON

```python
def get_product_or_404(db, product_id: int) -> Product:
    product = db.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail=f"Product {product_id} not found")
    return product
```

⚠️ `return` an error vs `raise` an error

`return HTTPException(404, "...")` does **not** send a 404 — the client gets `200 OK` with a body like `{"status_code":404,"detail":"...","headers":null}`, which looks like success to every frontend. Always `raise`. And never leak internals: `detail=str(e)` from a database exception can expose table names and SQL to attackers.

🔬 Go Deeper — custom exceptions + one exception handler

Remember Class 9's `AppError` tree? Keep your business logic free of HTTP details and translate once, at the edge:

PYTHON

```python
from fastapi import Request
from fastapi.responses import JSONResponse

class AppError(Exception):
    status_code = 400
    def __init__(self, message: str):
        self.message = message

class NotFoundError(AppError):
    status_code = 404

class ConflictError(AppError):
    status_code = 409

@app.exception_handler(AppError)                # catches AppError AND every subclass
def handle_app_error(request: Request, exc: AppError):
    return JSONResponse(status_code=exc.status_code, content={"detail": exc.message})

# anywhere in your code:   raise NotFoundError("Product 99 not found")   -> 404 JSON
```

28 Organising Code with `APIRouter`

One `main.py` with 30 endpoints is the "UserManager with 30 methods" smell from Class 9. An **APIRouter** is a mini-app for one resource; `main.py` just plugs routers in.

PYTHON - APP/ROUTERS/PRODUCTS.PY

```python
from fastapi import APIRouter, HTTPException, status
from sqlalchemy import select

from app.database import SessionDep
from app.models import Product
from app.schemas import ProductCreate, ProductOut

router = APIRouter(prefix="/products", tags=["Products"])   # every path starts with /products


@router.post("", response_model=ProductOut, status_code=status.HTTP_201_CREATED)
def create_product(data: ProductCreate, db: SessionDep):
    ...                                     # same body as Section 25


@router.get("", response_model=list[ProductOut])
def list_products(db: SessionDep):
    ...


@router.get("/{product_id}", response_model=ProductOut)    # full path: /products/{product_id}
def read_product(product_id: int, db: SessionDep):
    ...
```

PYTHON - APP/MAIN.PY

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI

from app.database import Base, engine
from app.routers import products, users


@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield

app = FastAPI(title="Shop API", lifespan=lifespan)
app.include_router(products.router)
app.include_router(users.router)
```

Run it from the project root with `fastapi dev app/main.py`. In `/docs`, endpoints are now grouped under **Products** and **Users** headings thanks to `tags`.

PRACTICE 32

1. Move your Student code into `app/` with `database.py`, `models.py`, `schemas.py` and `routers/students.py`.
2. Build `POST /students` (201, 409 on duplicate email), `GET /students`, `GET /students/{id}` (404 when missing).
3. Create 3 students in Swagger, stop the server, start it again, and confirm they are still there.
4. Try `return HTTPException(...)` instead of `raise`. Look at the status code you get, then fix it.

PART E · CLASS 33

CRUD Endpoints — Part 2: Update, Delete, Paginate, Filter

29 PUT vs PATCH — Two Ways to Update

Both change an existing record. The difference is **how much** the client must send:

	`PUT` — replace	`PATCH` — modify
Meaning	"Here is the **complete** new version"	"Change **only** these fields"
Body	All required fields (reuse `ProductCreate`)	Any subset (`ProductUpdate`, all optional)
Missing field	`422` — required	Left unchanged
Typical UI	"Edit product" form that submits everything	A toggle, an inline price edit, "mark as done"

PYTHON - APP/SCHEMAS.PY (ADD)

```python
class ProductUpdate(BaseModel):            # every field optional, same rules as Create
    name: str | None = Field(default=None, min_length=2, max_length=100)
    price: float | None = Field(default=None, gt=0)
    stock: int | None = Field(default=None, ge=0)
    description: str | None = None
    is_active: bool | None = None
```

PYTHON - APP/ROUTERS/PRODUCTS.PY (ADD)

```python
@router.put("/{product_id}", response_model=ProductOut)
def replace_product(product_id: int, data: ProductCreate, db: SessionDep):
    product = get_product_or_404(db, product_id)
    for field, value in data.model_dump().items():          # EVERY field is overwritten
        setattr(product, field, value)
    db.commit()
    db.refresh(product)
    return product


@router.patch("/{product_id}", response_model=ProductOut)
def update_product(product_id: int, data: ProductUpdate, db: SessionDep):
    product = get_product_or_404(db, product_id)
    changes = data.model_dump(exclude_unset=True)           # ONLY fields the client sent
    for field, value in changes.items():
        setattr(product, field, value)                      # product.price = 900  etc.
    db.commit()
    db.refresh(product)
    return product
```

`setattr(obj, "price", 900)` is the same as `obj.price = 900`, but the attribute name comes from a variable — perfect for looping over a dict of changes.

⚠️ The `exclude_unset` bug — the most common PATCH mistake

The client sends `{"price": 900}`. Look at the difference:

PYTHON

```python
data.model_dump()
# {'name': None, 'price': 900.0, 'stock': None, 'description': None, 'is_active': None}
#   -> your loop sets name = None, stock = None ...  product destroyed (or a NOT NULL crash)

data.model_dump(exclude_unset=True)
# {'price': 900.0}          -> only price changes. Correct.
```

`exclude_unset=True` means "fields the client did not include". It still keeps a field the client *explicitly* set to `null` — so `{"description": null}` correctly clears the description.

Unique fields on update

If an update can change a unique field (SKU, email), repeat the duplicate check — but exclude the record itself: `select(Product).where(Product.sku == new_sku, Product.id != product_id)`. Otherwise saving a product with its own unchanged SKU reports a false 409.

30 DELETE — Remove a Record

PYTHON

```python
@router.delete("/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_product(product_id: int, db: SessionDep):
    product = get_product_or_404(db, product_id)    # deleting a missing id -> 404
    db.delete(product)
    db.commit()
    # no return: 204 means "success, and the body is intentionally empty"
```

Request	Response
`DELETE /products/3` **(exists)**	`204`, empty body
`DELETE /products/3` **again**	`404` `{"detail": "Product 3 not found"}`
`GET /products/3` **afterwards**	`404`

🔬 **Go Deeper — soft delete**

Real systems rarely destroy rows: an order history that points to a deleted product breaks. Instead they set a flag — `product.is_active = False` (or `deleted_at = now`) — and every list query adds `.where(Product.is_active == True)`. The data is hidden, not lost, and can be restored. Banks and e-commerce sites almost always soft-delete.

31 Pagination — `skip` & `limit`

`GET /products` returning **all** 200,000 rows would be slow for the database, heavy for the network and unreadable for the user. **Pagination** returns one page at a time. The simplest style is **offset pagination**:

Param	Meaning	SQL
`skip`	How many rows to jump over	`OFFSET`
`limit`	How many rows to return (with a **maximum**)	`LIMIT`

Page *n* with page size *s*: `skip = (n - 1) * s`, `limit = s` → **page 3 of 10:** `?skip=20&limit=10`

PYTHON

```python
from typing import Annotated
from fastapi import Query


@router.get("", response_model=list[ProductOut])
def list_products(
    db: SessionDep,
    skip: Annotated[int, Query(ge=0)] = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,      # a client can never ask for 1 million
):
    stmt = select(Product).order_by(Product.id).offset(skip).limit(limit)
    return db.scalars(stmt).all()
```

⚠️ **Always `order_by` when paginating**

Without an explicit order, the database may return rows in any order — and a different order on the next query. Page 2 can then repeat items from page 1 or skip some entirely. Paginate on a stable order, usually `id`.

Returning the total — a page envelope

A frontend needs to know how many pages exist ("Page 3 of 17"). Return the items inside an envelope with the total:

PYTHON

```python
from sqlalchemy import func


class ProductPage(BaseModel):
    total: int                  # rows matching the filters (before skip/limit)
    skip: int
    limit: int
    items: list[ProductOut]


@router.get("", response_model=ProductPage)
def list_products(db: SessionDep,
                  skip: Annotated[int, Query(ge=0)] = 0,
                  limit: Annotated[int, Query(ge=1, le=100)] = 20):
    stmt = select(Product)
    total = db.scalar(select(func.count()).select_from(stmt.subquery()))   # COUNT(*) of the query
    items = db.scalars(stmt.order_by(Product.id).offset(skip).limit(limit)).all()
    return {"total": total, "skip": skip, "limit": limit, "items": items}
```

RESPONSE - GET /PRODUCTS?SKIP=20&LIMIT=2

```
{
  "total": 57,
  "skip": 20,
  "limit": 2,
  "items": [
    {"id": 21, "name": "USB Hub", "price": 1200.0, "...": "..."},
    {"id": 22, "name": "Webcam", "price": 3500.0, "...": "..."}
  ]
}
```

32 Filtering, Searching & Sorting with Query Params

Class 29 filtered a Python list with `if`s. With a database, you build the query **step by step** — each optional query param adds one `.where()` only if the client sent it. Nothing runs until `db.scalars()`.

PYTHON - APP/ROUTERS/PRODUCTS.PY - THE FULL LIST ENDPOINT

```python
from typing import Annotated, Literal
from fastapi import HTTPException, Query
from sqlalchemy import func, select

SORT_COLUMNS = {"id": Product.id, "name": Product.name, "price": Product.price,
                "created_at": Product.created_at}


@router.get("", response_model=ProductPage)
def list_products(
    db: SessionDep,
    q: Annotated[str | None, Query(min_length=2, description="search in name")] = None,
    min_price: Annotated[float | None, Query(ge=0)] = None,
    max_price: Annotated[float | None, Query(ge=0)] = None,
    in_stock: bool | None = None,
    sort_by: Literal["id", "name", "price", "created_at"] = "id",
    order: Literal["asc", "desc"] = "asc",
```

```
    skip: Annotated[int, Query(ge=0)] = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
):
    if min_price is not None and max_price is not None and min_price > max_price:
        raise HTTPException(400, "min_price cannot be greater than max_price")

    stmt = select(Product).where(Product.is_active == True)        # base query
    if q:
        stmt = stmt.where(Product.name.ilike(f"%{q}%"))            # case-insensitive contains
    if min_price is not None:
        stmt = stmt.where(Product.price >= min_price)
    if max_price is not None:
        stmt = stmt.where(Product.price <= max_price)
    if in_stock is True:
        stmt = stmt.where(Product.stock > 0)
    elif in_stock is False:
        stmt = stmt.where(Product.stock == 0)

    total = db.scalar(select(func.count()).select_from(stmt.subquery()))

    column = SORT_COLUMNS[sort_by]
    stmt = stmt.order_by(column.desc() if order == "desc" else column.asc(), Product.id)
    items = db.scalars(stmt.offset(skip).limit(limit)).all()
    return {"total": total, "skip": skip, "limit": limit, "items": items}
```

Request	Becomes
`/products`	active products, by id, first 20
`/products?q=key`	`... AND lower(name) LIKE lower('%key%')`
`/products?min_price=500&max_price=2000&in_stock=true`	three extra `WHERE` conditions, joined with `AND`
`/products?sort_by=price&order=desc&limit=5`	5 most expensive
`/products?sort_by=password`	`422` — not one of the allowed `Literal` values
`/products?min_price=3000&max_price=100`	`400` — your business rule

Three details worth copying

1. `in_stock: bool | None = None` has *three* states — not sent, `true`, `false` — so check `is True` / `is False`, not just truthiness. **2.** `Literal[...]` whitelists sort columns, so a client can never sort by a column you did not choose. **3.** The tie-breaker `Product.id` in `order_by` keeps pages stable when two products have the same price.

⚠️ Never build SQL with f-strings

`f"SELECT * FROM products WHERE name LIKE '%{q}%'"` lets a client send `q=' OR 1=1 --` and read your whole table — **SQL injection**. SQLAlchemy's `.where()` / `.ilike()` send values as *parameters*, never as SQL text. The f-string inside `.ilike(f"%{q}%")` is safe because it only builds the *value*, which is still sent as a parameter. (Class 9's `QueryBuilder` was a toy for exactly this reason.)

PRACTICE 33

1. Add `PUT`, `PATCH` and `DELETE` to your Student API. Prove the difference: PATCH with `{"cgpa": 3.9}`, then PUT with only `{"cgpa": 3.9}` and read the 422.
2. Temporarily remove `exclude_unset=True`, PATCH one field, and look at what happened to the other fields. Put it back.
3. Make `GET /students` return a page envelope with `total`. Seed 25 students and walk through pages with `skip` / `limit`.
4. Add filters `dept`, `min_cgpa`, `q` (name search), and `sort_by` = `name|cgpa` with `order`.

PART F

Review — Mistakes, Concept Map, Full Example

33 Common Mistakes

Wrong ✗	Right ✓
`pip install fastapi` then "uvicorn: command not found"	`pip install "fastapi[standard]"` inside an activated venv
`/users/{id}` declared before `/users/me`	Fixed paths first, parameter paths after
`data: dict` as the request body	A Pydantic model: `data: ProductCreate`
Returning the ORM user object with no `response_model`	`response_model=UserOut` — never leak passwords
Using `ProductCreate` for PATCH	A separate `ProductUpdate` with all fields optional
`data.model_dump()` in PATCH	`data.model_dump(exclude_unset=True)`
Forgetting `from_attributes=True` on Out schemas	`model_config = ConfigDict(from_attributes=True)`
`db.add(obj)` without `db.commit()`	add → commit → refresh
Opening `SessionLocal()` inside every endpoint	`db: SessionDep` via `get_db()` with `yield`
`return {"error": "not found"}` with status 200	`raise HTTPException(404, ...)`
`POST` returning 200	`status_code=status.HTTP_201_CREATED`
List endpoint with no `limit` maximum	`Query(ge=1, le=100)` on `limit`
Paginating without `order_by`	`.order_by(Model.id)` before `.offset().limit()`
Sync session inside `async def`	`def` with sync sessions; `async def` only with `AsyncSession`
Committing `shop.db` and `.venv/` to Git	Add both to `.gitignore`; commit `requirements.txt`

34 Concept Map

🚀 Class 29

`FastAPI()` · `@app.get`

path params → *which one*

query params → *which ones, how*

`Query(ge, le)`

`/docs` · OpenAPI

📐 Class 30

`BaseModel` = body

`Field` · validators

`X | None = None`

`response_model`

Create / Update / Out

🗄️ Class 31

engine · session · Base

`Mapped` · `mapped_column`

model ≠ schema

`get_db()` + `yield`

`from_attributes`

🔁 Classes 32–33

POST 201 · GET 200

PUT / PATCH · DELETE 204

`HTTPException` 404/409

`exclude_unset`

skip · limit · where · order_by

HTTP request JSON + params → **Pydantic schema** validates (422) → **Endpoint** business rules (400/404/409) → **SQLAlchemy model** via session from `get_db` → **Database** SQLite / Postgres → **response_model** filters → JSON

> **One sentence to remember:** a FastAPI endpoint is a function whose type hints are rules — **schemas** guard what comes in and what goes out, **models** describe how it is stored, **get_db** hands each request its own session, and **status codes** tell the client exactly what happened.

35 Putting It Together — Task Tracker API

Everything from Classes 29–33 in one runnable file: model, schemas (Create / Update / Out), `get_db` dependency, lifespan, 201/204/404/422, PATCH with `exclude_unset`, filtering, search and pagination. In a real project you would split it into the files from Section 3 — here it is kept in one file so you can read it top to bottom.

PYTHON - TASKS_APP.PY

```python
from contextlib import asynccontextmanager
from datetime import datetime, timezone
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query, status
from pydantic import BaseModel, ConfigDict, Field
from sqlalchemy import String, create_engine, select
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column, sessionmaker

# ---------- database ----------
engine = create_engine("sqlite:///./tasks.db", connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False)


class Base(DeclarativeBase):
    pass


class Task(Base):                                   # MODEL = how it is stored
```

```python
    __tablename__ = "tasks"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))
    priority: Mapped[int] = mapped_column(default=3)
    done: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(default=lambda: datetime.now(timezone.utc))


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

SessionDep = Annotated[Session, Depends(get_db)]


# ---------- schemas ----------
class TaskCreate(BaseModel):                        # SCHEMA = how it travels
    title: str = Field(min_length=3, max_length=100)
    priority: int = Field(default=3, ge=1, le=5)


class TaskUpdate(BaseModel):                        # every field optional -> PATCH
    title: str | None = Field(default=None, min_length=3, max_length=100)
    priority: int | None = Field(default=None, ge=1, le=5)
    done: bool | None = None


class TaskOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    title: str
    priority: int
    done: bool


# ---------- app ----------
@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield

app = FastAPI(title="Task Tracker", lifespan=lifespan)


def get_task_or_404(db: Session, task_id: int) -> Task:
    task = db.get(Task, task_id)
    if task is None:
        raise HTTPException(status_code=404, detail=f"Task {task_id} not found")
    return task


@app.post("/tasks", response_model=TaskOut, status_code=status.HTTP_201_CREATED)
def create_task(data: TaskCreate, db: SessionDep):
    task = Task(**data.model_dump())
    db.add(task)
    db.commit()
    db.refresh(task)
```

```
    return task


@app.get("/tasks", response_model=list[TaskOut])
def list_tasks(
    db: SessionDep,
    done: bool | None = None,
    min_priority: Annotated[int | None, Query(ge=1, le=5)] = None,
    q: Annotated[str | None, Query(min_length=2)] = None,
    skip: Annotated[int, Query(ge=0)] = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 10,
):
    stmt = select(Task)
    if done is not None:
        stmt = stmt.where(Task.done == done)
    if min_priority is not None:
        stmt = stmt.where(Task.priority >= min_priority)
    if q:
        stmt = stmt.where(Task.title.ilike(f"%{q}%"))
    stmt = stmt.order_by(Task.id).offset(skip).limit(limit)
    return db.scalars(stmt).all()


@app.get("/tasks/{task_id}", response_model=TaskOut)
def read_task(task_id: int, db: SessionDep):
    return get_task_or_404(db, task_id)


@app.patch("/tasks/{task_id}", response_model=TaskOut)
def update_task(task_id: int, data: TaskUpdate, db: SessionDep):
    task = get_task_or_404(db, task_id)
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(task, field, value)
    db.commit()
    db.refresh(task)
    return task


@app.delete("/tasks/{task_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(task_id: int, db: SessionDep):
    task = get_task_or_404(db, task_id)
    db.delete(task)
    db.commit()
```

Run `fastapi dev tasks_app.py` on a fresh database and send these requests from `/docs`, in order:

#	Request	Status	Response body
1	`POST` `/tasks` `{"title": "Write notes", "priority": 5}`	201	`{"id": 1, "title": "Write notes", "priority": 5, "done": false}`
2	`POST` `/tasks` `{"title": "Fix login bug", "priority": 4}`	201	`{"id": 2, ... "done": false}`
3	`POST` `/tasks` `{"title": "Buy milk"}`	201	`{"id": 3, "title": "Buy milk", "priority": 3, "done": false}`

4	PATCH `/tasks/2` `{"done": true}`	200	`{"id": 2, "title": "Fix login bug", "priority": 4, "done": true}`
5	GET `/tasks?done=false&min_priority=4`	200	only task 1
6	GET `/tasks?skip=1&limit=1`	200	only task 2
7	GET `/tasks?q=bug`	200	only task 2
8	DELETE `/tasks/3`	204	(empty)
9	GET `/tasks/3`	404	`{"detail": "Task 3 not found"}`
10	POST `/tasks` `{"title": "Hi", "priority": 9}`	422	two errors: `["body","title"]` and `["body","priority"]`

What to notice

The endpoints never check types or ranges — schemas did it (request 10). The endpoints never open or close a session — `get_db` did it. `TaskOut` hides `created_at` even though the model has it. PATCH changed only `done` (request 4). Every function has one job — exactly the design habits from Class 9.

CAPSTONE PROJECT · CLASSES 29–33

Build it, don't just read it

This project uses **every** idea from the five classes. You get a **specification**, **seed data** and the **exact expected behaviour** — no solution code. Your job is to design and write the code so that every request in the test table returns exactly what is shown. Read the whole spec twice before you type anything. Build it **class by class** using the milestones — after Class 29 you can already start.

🎯 CAPSTONE PROJECT

BookNest — Online Bookstore API

Routes · Path & Query Params · Pydantic Schemas · SQLAlchemy · get_db() · Full CRUD · Pagination · Filtering

A small online bookstore in Dhaka wants a backend for its new mobile app. The app must list books with filters and pages, show one book with its author, let staff add and edit books and authors, and adjust stock when books are sold or restocked. Bad data must never reach the database, and every failure must return the right status code with a clear message.

TECH STACK — REQUIRED

- Python 3.10+, `fastapi[standard]`, `sqlalchemy` 2.x (new style: `DeclarativeBase`, `Mapped`, `select()`), Pydantic v2.
- SQLite database file `booknest.db`, **sync** sessions, `def` endpoints.
- Tables created in a **lifespan** function. DB session provided only through a `get_db()` dependency with `yield`.

PROJECT STRUCTURE — REQUIRED

- `app/main.py` — creates the app, lifespan, includes the two routers, health endpoint.
- `app/database.py` — engine, `SessionLocal`, `Base`, `get_db()`, `SessionDep`.
- `app/models.py` — `Author` and `Book` models. `app/schemas.py` — every Pydantic schema.
- `app/routers/authors.py` and `app/routers/books.py` — one `APIRouter` each, with `prefix` and `tags`.
- `scripts/seed.py` — inserts the seed data (below) **only if the tables are empty**.
- `requirements.txt`, `.gitignore` (must ignore `.venv/`, `__pycache__/`, `*.db`), `README.md`.

📦 PART 1 OF 4

Database Models

MODEL Author — table "authors"

Column	Type	Rules
`id`	int	primary key
`name`	str (60)	required
`country`	str (60) | None	nullable
`bio`	str (500) | None	nullable
`created_at`	datetime	set automatically (UTC) when the row is created
`books`	relationship	list of this author's `Book` objects (`back_populates`)

MODEL Book — table "books"

Column	Type	Rules
`id`	int	primary key
`title`	str (150)	required, indexed
`isbn`	str (13)	**unique**; stored as 13 digits, no hyphens
`author_id`	int	`ForeignKey("authors.id")`
`genre`	str	one of the genre values below
`price`	float	required
`stock`	int	default 0
`published_year`	int	required
`created_at` / `updated_at`	datetime	both set on create; `updated_at` refreshed on every PUT / PATCH / stock change
`author`	relationship	the `Author` object this book belongs to

📐 PART 2 OF 4

Pydantic Schemas

GENRE + AUTHOR SCHEMAS

- `Genre` — a `str` Enum: `fiction`, `non_fiction`, `science`, `history`, `programming`, `kids`.
- `AuthorCreate` — `name` 2–60 chars (required), `country` ≤ 60 chars (optional), `bio` ≤ 500 chars (optional).
- `AuthorUpdate` — same three fields, **all optional**, same rules.
- `AuthorOut` — `id, name, country, bio, created_at`. `AuthorBrief` — only `id, name` (used inside books).

BOOK SCHEMAS

- `BookCreate` — `title` 2–150 chars · `isbn` · `author_id` · `genre: Genre` · `price` > 0 and ≤ 50,000 · `stock` ≥ 0 (default 0) · `published_year` ≥ 1450 and **not in the future**.
- `isbn` rule (`@field_validator`): remove every `-`; the rest must be exactly 13 digits, otherwise reject. The validator **returns the cleaned value**, so `"978-0-13-235088-4"` becomes `"9780132350884"`.
- `published_year` "not in the future" must use the current year at runtime — not a hard-coded number.
- `BookUpdate` — every `BookCreate` field, all optional, same rules (including the ISBN cleaning).
- `BookOut` — `id, title, isbn, genre, price, stock, published_year, author: AuthorBrief, created_at, updated_at`. **No** `author_id` field — the nested author replaces it.
- `StockChange` — `change: int`, must not be 0 (positive = restock, negative = sale).
- `BookPage` / `AuthorPage` — `total, skip, limit, items`.

Part 3 of 4 — Endpoints

Method & path	Body	Success	Errors
`GET /`	—	200	—
`POST /authors`	`AuthorCreate`	201	422
`GET /authors`	—	200	422 (bad query params)
`GET /authors/{author_id}`	—	200	404
`PATCH /authors/{author_id}`	`AuthorUpdate`	200	404, 422
`DELETE /authors/{author_id}`	—	204	404, 409 (author still has books)
`GET /authors/{author_id}/books`	—	200	404, 422
`POST /books`	`BookCreate`	201	404 (author), 409 (ISBN), 422

GET	/books	—	200	400 (price range), 422
GET	/books/{book_id}	—	200	404
PUT	/books/{book_id}	BookCreate	200	404 (book / author), 409 (ISBN), 422
PATCH	/books/{book_id}	BookUpdate	200	404 (book / author), 409 (ISBN), 422
PATCH	/books/{book_id}/stock	StockChange	200	404, 409 (not enough stock), 422
DELETE	/books/{book_id}	—	204	404

Response shapes. `GET /` returns `{"status": "ok", "service": "BookNest"}`. Every single-book response is a `BookOut`; every single-author response is an `AuthorOut`; every list is a page envelope:

EXPECTED SHAPE - GET /BOOKS/2 (TIMESTAMPS WILL DIFFER)


```
{
  "id": 2,
  "title": "Clean Code",
  "isbn": "9780132350884",
  "genre": "programming",
  "price": 2800.0,
  "stock": 5,
  "published_year": 2008,
  "author": {"id": 2, "name": "Robert C. Martin"},
  "created_at": "2026-10-24T09:15:02.481233",
  "updated_at": "2026-10-24T09:15:02.481233"
}
```


EXPECTED SHAPE - GET /BOOKS?IN_STOCK=TRUE&LIMIT=2


```
{
  "total": 4,
  "skip": 0,
  "limit": 2,
  "items": [ {"id": 1, "title": "Moyurakkhi", "...": "..."}, {"id": 2, "title": "Clean Code", "...":
"..."} ]
}
```


Query parameters for `GET /books`

Param	Type & rule	Effect
`q`	str, min length 2, optional	Title contains `q`, case-insensitive
`genre`	`Genre`, optional	Exact genre (invalid value → 422)
`author_id`	int, optional	Books by that author
`min_price` / `max_price`	float ≥ 0, optional	Price range, inclusive. Both given and min > max → **400**

`in_stock`	bool, optional (3 states)	`true` → stock > 0 · `false` → stock = 0 · not sent → all
`sort_by`	`id | title | price | published_year`, default `id`	Sort column (anything else → 422)
`order`	`asc | desc`, default `asc`	Sort direction; ties always broken by `id` ascending
`skip` / `limit`	`skip` ≥ 0 (default 0) · `limit` 1–50 (default 10)	Pagination. `total` counts **all** matching books, before skip/limit

`GET /authors` supports `q` (name contains, case-insensitive), `country` (exact), `skip`, `limit` (same rules), sorted by `id`. `GET /authors/{author_id}/books` supports `skip` and `limit`, sorted by `id`.

Part 4 of 4 — Business rules & exact error messages

Situation	Status	`detail` — must match exactly
Book id does not exist (any book endpoint)	404	`Book {book_id} not found`
Author id does not exist — in a path, or as `author_id` in a book body	404	`Author {author_id} not found`
ISBN already used by another book (POST, PUT, PATCH)	409	`Book with ISBN {cleaned_isbn} already exists`
Deleting an author who still has books	409	`Author {author_id} still has {n} books`
Stock change would make stock negative	409	`Not enough stock: {current_stock} available`
`min_price` greater than `max_price`	400	`min_price cannot be greater than max_price`
Any schema / param validation failure	422	FastAPI's automatic error list

> **Rules that are easy to miss**
>
> **1.** On PUT/PATCH, a book keeping its *own* ISBN is not a duplicate. **2.** On PATCH, check the author / ISBN only if the client actually sent those fields. **3.** PATCH must change **only** the fields that were sent. **4.** A failed request must change nothing in the database. **5.** Check order for `POST /books` : schema (422) → author exists (404) → ISBN unique (409).

Seed data — `scripts/seed.py` must insert exactly this, in this order

SEED DATA - AUTHORS (IDS 1-3) THEN BOOKS (IDS 1-6)

```
AUTHORS
  1  Humayun Ahmed        country: Bangladesh
  2  Robert C. Martin     country: USA
  3  Yuval Noah Harari    country: Israel

BOOKS   title               isbn (as given)     author  genre        price  stock  year
  1     Moyurakkhi          978-984-000-0011    1       fiction       350    12     1990
  2     Clean Code          978-0132350884      2       programming  2800     5     2008
  3     Clean Architecture  978-0134494166      2       programming  3100     0     2017
  4     Sapiens             978-0062316097      3       history      1450     8     2014
  5     Homo Deus           978-0062464316      3       history      1600     3     2016
  6     Debi                978-984-000-0028    1       fiction       300     0     1985
```

The two Bangla-book ISBNs are made-up test values. Seed through your own schemas/validation so the hyphens are cleaned exactly as they would be for an API request.

Expected behaviour — run in this order on a freshly seeded database

Delete `booknest.db`, start the app, run `python scripts/seed.py`, then send these requests from `/docs` (or write them as a test file — see the stretch goals). Every status and every listed value must match.

#	Request	Status	Must return
1	GET `/`	200	`{"status": "ok", "service": "BookNest"}`
2	GET `/books`	200	`total` 6, titles in id order 1→6
3	GET `/books?genre=programming`	200	`total` 2: Clean Code, Clean Architecture
4	GET `/books?genre=history&sort_by=price&order=desc`	200	Homo Deus, Sapiens
5	GET `/books?in_stock=true&limit=2`	200	`total` 4; items: Moyurakkhi, Clean Code
6	GET `/books?in_stock=true&skip=2&limit=2`	200	`total` 4; items: Sapiens, Homo Deus
7	GET `/books?min_price=1000&max_price=2000`	200	`total` 2: Sapiens, Homo Deus
8	GET `/books?min_price=3000&max_price=1000`	400	`min_price cannot be greater than max_price`
9	GET `/books?q=CLEAN`	200	`total` 2 (case-insensitive)
10	GET `/books?author_id=1&sort_by=published_year`	200	Debi, Moyurakkhi

11	GET `/books?limit=100`	422	error at `["query", "limit"]`
12	GET `/books?sort_by=password`	422	error at `["query", "sort_by"]`
13	GET `/books/2`	200	`isbn` `"9780132350884"`, `author` `{"id": 2, "name": "Robert C. Martin"}`
14	GET `/books/99`	404	`Book 99 not found`
15	POST `/books` — Clean Code's data again but `"isbn": "978-0-13-235088-4"`	409	`Book with ISBN 9780132350884 already exists`
16	POST `/books` — valid book, new ISBN, `"author_id": 99`	404	`Author 99 not found`
17	POST `/books` — `"isbn": "12-34"`, `"price": -5`	422	errors at `["body","isbn"]` and `["body","price"]`
18	POST `/books` — `"published_year": 2090`	422	error at `["body","published_year"]`
19	PATCH `/books/3` `{"stock": 10}`	200	`stock` 10, `price` still 3100.0, `updated_at` changed
20	PATCH `/books/3` `{"isbn": "9780132350884"}`	409	`Book with ISBN 9780132350884 already exists`
21	PUT `/books/5` — body without `price`	422	error at `["body","price"]`
22	PUT `/books/5` — full body, same ISBN, `"price": 1750`	200	`price` 1750.0 (own ISBN is not a duplicate)
23	PATCH `/books/4/stock` `{"change": -20}`	409	`Not enough stock: 8 available`
24	PATCH `/books/4/stock` `{"change": -3}`	200	`stock` 5
25	PATCH `/books/4/stock` `{"change": 0}`	422	error at `["body","change"]`
26	POST `/authors` `{"name": "A"}`	422	error at `["body","name"]`
27	POST `/authors` `{"name": "Muhammed Zafar Iqbal", "country": "Bangladesh"}`	201	`id` 4
28	GET `/authors?country=Bangladesh`	200	`total` 2: Humayun Ahmed, Muhammed Zafar Iqbal
29	DELETE `/authors/2`	409	`Author 2 still has 2 books`
30	DELETE `/authors/4`	204	empty body
31	DELETE `/books/6`	204	empty body
32	GET `/books/6`	404	`Book 6 not found`

33	`GET` `/authors/1/books`	**200**	`total` 1: Moyurakkhi
34	`GET` `/authors/99/books`	**404**	`Author 99 not found`
35	`GET` `/books?in_stock=false`	**200**	`total` 0 (book 3 was restocked, book 6 deleted)

Milestones — build it class by class

After class	You should have…
29	Repo, venv, `requirements.txt`, `.gitignore`. `GET /` health check. An **in-memory** list of the 6 seed books with `GET /books` (filters `genre`, `max_price`) and `GET /books/{book_id}`. Everything visible in `/docs`.
30	`app/schemas.py` complete — every schema from Part 2 with all validation rules. `POST /books` and `POST /authors` still in memory but validated; requests 17, 18, 26 already behave correctly.
31	`database.py` + `models.py` with the foreign key and relationships, tables created in lifespan, `get_db()` wired in, `scripts/seed.py` working. Data survives a restart.
32	Two routers. All POST and GET endpoints on the database with 201 / 404 / 409 and the exact messages. Nested `author` in `BookOut`.
33	PUT, PATCH, stock change, DELETE, page envelopes, every filter and sort. All 35 requests in the table pass.

> **Before you submit**
>
> Run all 35 requests on a fresh database — not one you have been testing on for a week. Search your code: `SessionLocal()` should appear only in `database.py` (and the seed script); `print` should not appear in any endpoint; no endpoint may return a raw dict for a book — always through `BookOut`. Open `/docs`: every endpoint is grouped under **Authors** or **Books** and every schema shows its rules.

🔬 Go Deeper — Stretch goals

1. **Automated tests:** turn the 35-row table into `tests/test_api.py` with `fastapi.testclient.TestClient` and `pytest`. Use `app.dependency_overrides[get_db]` to point the tests at a separate test database.
2. **Soft delete:** add `is_active` to `Book`; `DELETE` sets it to `False`, and every list and detail endpoint hides inactive books.
3. **Custom exception tree:** replace the scattered `HTTPException`s with `NotFoundError` / `ConflictError` (Section 27) and one exception handler.
4. **Async version:** a second branch using `AsyncSession`, `aiosqlite` and `async def` endpoints. The 35 requests must still pass.
5. **Stats endpoint:** `GET /books/stats` returning total books, total stock value (`price × stock`) and the count per genre — computed with SQL (`func.sum`, `group_by`), not Python loops. Mind the route order!

✅ SUBMISSION CHECKLIST

1. GitHub repo named `booknest-api` with the required structure, pushed with clear commit messages (at least one commit per milestone).
2. `README.md`: what the project is, how to install, how to seed, how to run, and a screenshot of your `/docs` page.
3. All 35 expected-behaviour requests pass on a freshly seeded database.
4. No `.venv/`, `__pycache__/` or `.db` files in the repo. `requirements.txt` is present.
5. Every schema and model has a one-line docstring. Link submitted.